

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ  
РОССИЙСКОЙ ФЕДЕРАЦИИ**

Федеральное государственное автономное образовательное учреждение  
высшего образования «Санкт-Петербургский политехнический  
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 Математика и компьютерные науки

Методы тестирования программного обеспечения

Отчет по лабораторной работе №2

«Тестирование методом черного и белого ящика»

Студент  
группы 5130201/30101

\_\_\_\_\_ Шишмаков В.О.

Преподаватель

\_\_\_\_\_ Курочкин М. А.

«\_\_\_\_\_» \_\_\_\_\_ 2026 г.

Санкт-Петербург, 2026

# Содержание

|                                                                            |           |
|----------------------------------------------------------------------------|-----------|
| <b>Введение</b>                                                            | <b>4</b>  |
| <b>1 Постановка задачи</b>                                                 | <b>5</b>  |
| <b>2 Описание методов тестирования</b>                                     | <b>6</b>  |
| 2.1 Метод черного ящика . . . . .                                          | 6         |
| 2.1.1 Эквивалентное разбиение . . . . .                                    | 6         |
| 2.1.2 Анализ граничных значений . . . . .                                  | 6         |
| 2.1.3 Причинно-следственные диаграммы . . . . .                            | 6         |
| 2.2 Метод белого ящика . . . . .                                           | 7         |
| 2.2.1 Покрытие операторов . . . . .                                        | 7         |
| 2.2.2 Покрытие решений . . . . .                                           | 7         |
| 2.2.3 Покрытие условий . . . . .                                           | 7         |
| 2.2.4 Покрытие решений и условий . . . . .                                 | 7         |
| 2.2.5 Комбинаторное покрытие условий . . . . .                             | 7         |
| <b>3 Тестирование программы №1</b>                                         | <b>8</b>  |
| 3.1 Описание программы . . . . .                                           | 8         |
| 3.2 Тестирование методом «черного» ящика . . . . .                         | 10        |
| 3.2.1 Разбиение на классы эквивалентности . . . . .                        | 10        |
| 3.2.2 Анализ граничных условий . . . . .                                   | 11        |
| 3.2.3 Причинно-следственная диаграмма . . . . .                            | 11        |
| 3.2.4 Результаты тестирования программы №1 методом чёрного ящика . . . . . | 13        |
| 3.3 Тестирование методом «белого» ящика . . . . .                          | 13        |
| 3.3.1 Покрытие операторов . . . . .                                        | 14        |
| 3.3.2 Покрытие решений . . . . .                                           | 14        |
| 3.3.3 Покрытие условий . . . . .                                           | 14        |
| 3.3.4 Покрытие решений и условий . . . . .                                 | 15        |
| 3.3.5 Комбинаторное покрытие условий . . . . .                             | 15        |
| 3.3.6 Результаты тестирования программы №1 методом белого ящика . . . . .  | 16        |
| <b>4 Тестирование программы №2</b>                                         | <b>17</b> |
| 4.1 Описание программы . . . . .                                           | 17        |
| 4.2 Тестирование методом «чёрного» ящика . . . . .                         | 19        |
| 4.2.1 Разбиение на классы эквивалентности . . . . .                        | 19        |
| 4.2.2 Анализ граничных условий . . . . .                                   | 20        |
| 4.2.3 Причинно-следственная диаграмма . . . . .                            | 21        |

|                                                                            |           |
|----------------------------------------------------------------------------|-----------|
| 4.2.4 Результаты тестирования программы №2 методом чёрного ящика . . . . . | 23        |
| 4.3 Тестирование методом белого ящика . . . . .                            | 23        |
| 4.3.1 Покрытие операторов . . . . .                                        | 24        |
| 4.3.2 Покрытие решений . . . . .                                           | 24        |
| 4.3.3 Покрытие условий . . . . .                                           | 25        |
| 4.3.4 Покрытие решений и условий . . . . .                                 | 25        |
| 4.3.5 Комбинаторное покрытие условий . . . . .                             | 25        |
| 4.3.6 Результаты тестирования программы №2 методом белого ящика . . . . .  | 26        |
| <b>5 Заключение</b>                                                        | <b>28</b> |

# Введение

Одним из важнейших этапов в процессе создания современного программного обеспечения является тестирование. Выделяют различные виды тестирования: модульное, интеграционное, функциональное, системное и приемочное. В рамках данной работы основное внимание уделяется модульному тестированию.

Модульное тестирование представляет собой базовый вид проверки, с которого, как правило, начинается оценка корректности функционирования программы. Оно предполагает проверку отдельных компонентов программного продукта: классов, модулей и функций. Основная цель модульного тестирования — обнаружение дефектов, то есть случаев, когда реальное поведение модуля расходится с его спецификацией.

В практике модульного тестирования применяются два основных подхода: метод «черного ящика» и метод «белого ящика».

Оба этих подхода будут рассмотрены и применены к обеим программам в данной работе.

# 1 Постановка задачи

**Цель:** провести тестирование двух программ методами черного и белого ящика.

**Задачи:**

1. Ознакомиться с подходами к модульному тестированию, а именно с методами черного и белого ящика.
2. Составить тесты для двух программ обоими методами и провести их.
3. Провести анализ результатов тестирования.

## **2 Описание методов тестирования**

### **2.1 Метод черного ящика**

Одним из ключевых подходов к тестированию является метод «черного ящика», известный также как тестирование, ориентированное на данные, либо тестирование на основе входных и выходных параметров. Согласно данному подходу, программа рассматривается как «черный ящик», внутренности и логика которого остаются скрыты и не принимаются во внимание. Основное внимание уделяется выявлению ситуаций, в которых реальная работа программы отклоняется от требований спецификации.

В рамках данного подхода необходимо выполнить проверку программы, используя такие техники, как разбиение входных данных на классы эквивалентности, анализ граничных значений и построение причинно-следственных диаграмм.

#### **2.1.1 Эквивалентное разбиение**

Качественно разработанный тест должен охватывать значительную часть прочих потенциальных тестов. В связи с этим целесообразно разделить все множество входных данных программы на ограниченное количество классов эквивалентности. Различают два типа классов: корректные классы, включающие допустимые входные значения, и некорректные классы, представляющие все остальные возможные состояния.

#### **2.1.2 Анализ граничных значений**

Граничные условия представляют собой ситуации, которые возникают в окрестностях предельных значений, определяющих классы эквивалентности. В отличие от эквивалентного разбиения, анализ граничных значений требует выбора элементов, позволяющих проверить именно каждую границу класса, и уделяет внимание как входным, так и выходным данным.

#### **2.1.3 Причинно-следственные диаграммы**

Причинно-следственная диаграмма — формализованный графический способ описания спецификации. Каждый узел принимает значение 0 или 1. Данный метод восполняет недостаток подходов, основанных на анализе граничных значений и разбиении на классы эквивалентности, не учитывающих комбинации входных условий.

## **2.2 Метод белого ящика**

В рамках подхода «белого ящика» предоставляется возможность изучать внутреннее устройство программы. Специалист по тестированию выбирает входные данные, анализируя логическую структуру программы.

### **2.2.1 Покрытие операторов**

Минимальным требованием является выполнение каждого оператора программы как минимум однократно.

### **2.2.2 Покрытие решений**

Необходимо подобрать такой набор тестов, при котором каждое логическое условие хотя бы один раз принимает значение «истина» и хотя бы один раз — «ложь».

### **2.2.3 Покрытие условий**

Этот критерий требует, чтобы набор тестов обеспечивал для каждого элементарного условия, входящего в состав составного условия, выполнение как минимум один раз значения «истина» и один раз значения «ложь».

### **2.2.4 Покрытие решений и условий**

Набор тестов считается достаточным, если каждое элементарное условие принимает оба возможных значения, каждый возможный результат решения достигается минимум один раз и каждая точка входа активизируется хотя бы однократно.

### **2.2.5 Комбинаторное покрытие условий**

Данный критерий предполагает формирование такого набора тестов, который обеспечивает проверку всех возможных комбинаций значений элементарных условий в каждом логическом решении.

## 3 Тестирование программы №1

### 3.1 Описание программы

**Автор:** Митяев Олег.

**Название:** Проверка простоты числа.

**Дано:**

- $n$  — целое число.

**Требуется:** определить, является ли число  $n$  простым.

**Ограничения:**

1.  $n$  вводится как целое число.
2. При некорректном вводе выводится сообщение **ошибка ввода** и выполняется повтор ввода.
3. Числа  $n \leq 1$  считаются непростыми.
4. Проверка простоты выполняется перебором делителей до  $[\sqrt{n}]$ .

**Спецификация:**

| Входные данные | Ожидаемый результат                            | Статус       | Комментарий                                   |
|----------------|------------------------------------------------|--------------|-----------------------------------------------|
| 7              | вывод: простое                                 | Корректно    | Обычный случай (простое число).               |
| 4              | вывод: не простое                              | Корректно    | Обычный случай (составное число).             |
| 2              | вывод: простое                                 | Корректно    | Граница: минимальное простое.                 |
| 1              | вывод: не простое                              | Корректно    | Граница: $n \leq 1$ непростое.                |
| -5             | вывод: не простое                              | Корректно    | Отрицательные числа непростые.                |
| abc            | вывод: ошибка ввода; ожидание повторного ввода | Ошибка ввода | Проверка обработки некорректного ввода.       |
| 80 12          | вывод: ошибка ввода; ожидание повторного ввода | Ошибка ввода | Обработка некорректного ввода (через пробел). |
| 3.45           | вывод: ошибка ввода; ожидание повторного ввода | Ошибка ввода | Обработка некорректного ввода (не целое).     |



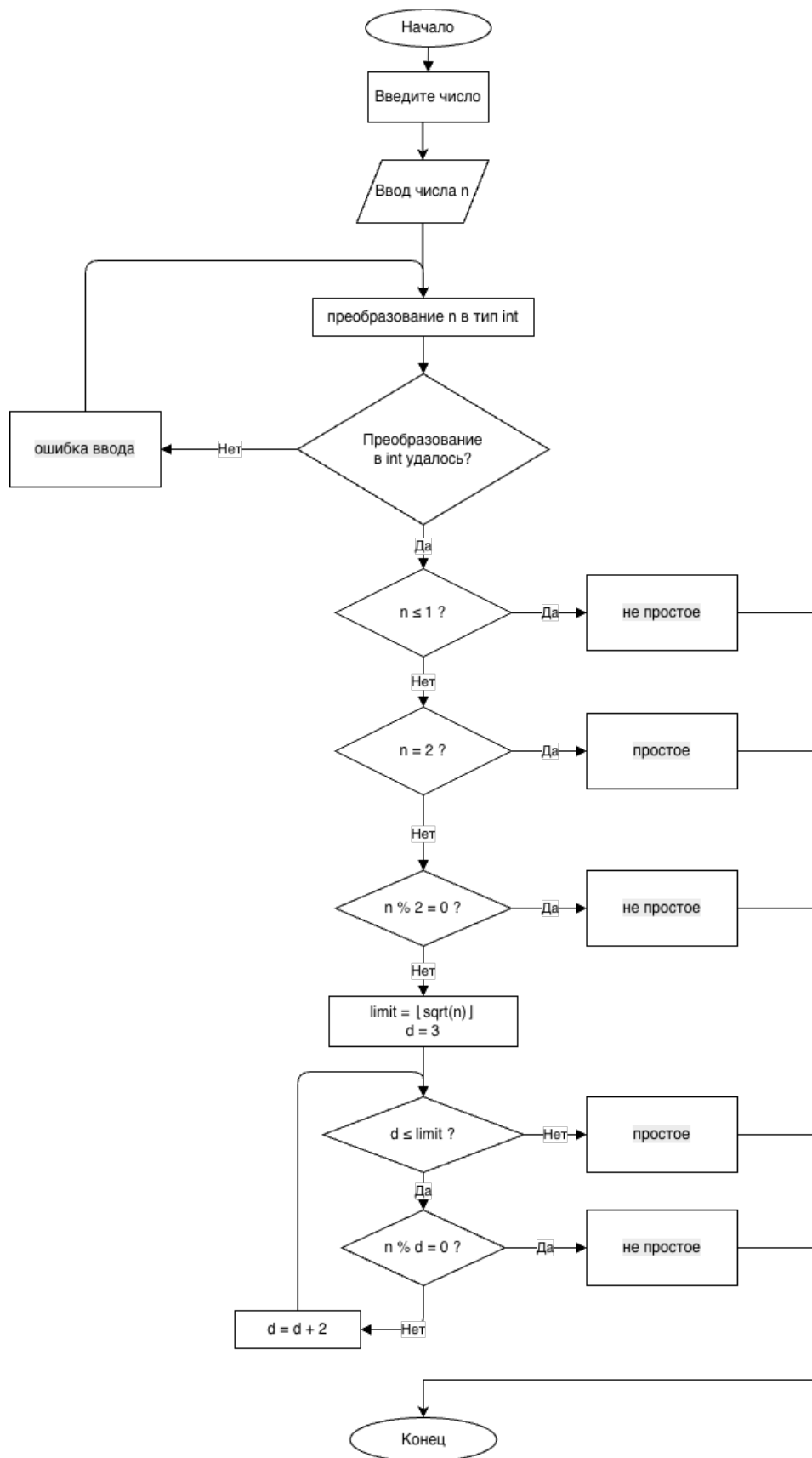


Рис. 1: Блок-схема программы №1

## 3.2 Тестирование методом «черного» ящика

### 3.2.1 Разбиение на классы эквивалентности

Составлено разбиение на классы эквивалентности, исходя из ограничений для тестируемой программы.

Таблица 1: Классы эквивалентности для  $n$

| Входное условие | Допустимые классы            | Недопустимые классы               |
|-----------------|------------------------------|-----------------------------------|
| Целое число $n$ | Отрицательное число (1)      | Нецелое вещественное число (8)    |
|                 | Ноль (2)                     | Строка (буквы) (9)                |
|                 | Единица (3)                  | Несколько чисел через пробел (10) |
|                 | Минимальное простое (4)      | Пустая строка (11)                |
|                 | Четное число $> 2$ (5)       | Спецсимволы / смешанный ввод (12) |
|                 | Нечетное простое число (6)   |                                   |
|                 | Нечетное составное число (7) |                                   |

В таблице 2 представлены тесты, покрывающие все классы эквивалентности.

Таблица 2: Тесты, покрывающие классы эквивалентности

| N  | $n$  | Класс эквивалентности | Ожидаемый результат          | Фактический результат        | Результат |
|----|------|-----------------------|------------------------------|------------------------------|-----------|
| 1  | -5   | (1)                   | Число не простое             | Число не простое             | Пройден   |
| 2  | 0    | (2)                   | Число не простое             | Число не простое             | Пройден   |
| 3  | 1    | (3)                   | Число не простое             | Число не простое             | Пройден   |
| 4  | 2    | (4)                   | Число простое                | Число простое                | Пройден   |
| 5  | 8    | (5)                   | Число не простое             | Число не простое             | Пройден   |
| 6  | 17   | (6)                   | Число простое                | Число простое                | Пройден   |
| 7  | 15   | (7)                   | Число не простое             | Число не простое             | Пройден   |
| 8  | 7.5  | (8)                   | Ошибка ввода; повторный ввод | Ошибка ввода; повторный ввод | Пройден   |
| 9  | abc  | (9)                   | Ошибка ввода; повторный ввод | Ошибка ввода; повторный ввод | Пройден   |
| 10 | 12 5 | (10)                  | Ошибка ввода; повторный ввод | Ошибка ввода; повторный ввод | Пройден   |
| 11 | «»   | (11)                  | Ошибка ввода; повторный ввод | Ошибка ввода; повторный ввод | Пройден   |
| 12 | 10a? | (12)                  | Ошибка ввода; повторный ввод | Ошибка ввода; повторный ввод | Пройден   |

Все 12 тестов прошли успешно.

### 3.2.2 Анализ граничных условий

Исходя из ограничений данной программы выделяем следующие граничные значения с точки зрения алгоритма:

1. Переход от наибольшего непростого к наименьшему простому (1, 2).
2. Переход между наименьшим четным составным (3, 4, 5).
3. Делитель ровно на границе  $\sqrt{n}$ :  $\sqrt{9} = 3$  (8, 9, 10).

Таблица 3: Проверка граничных условий

| N | n  | Ожидаемый результат | Фактический результат | Результат |
|---|----|---------------------|-----------------------|-----------|
| 1 | 1  | Число не простое    | Число не простое      | Пройден   |
| 2 | 2  | Число простое       | Число простое         | Пройден   |
| 3 | 3  | Число простое       | Число простое         | Пройден   |
| 4 | 4  | Число не простое    | Число не простое      | Пройден   |
| 5 | 5  | Число простое       | Число простое         | Пройден   |
| 6 | 8  | Число не простое    | Число не простое      | Пройден   |
| 7 | 9  | Число не простое    | Число не простое      | Пройден   |
| 8 | 10 | Число не простое    | Число не простое      | Пройден   |

При проведении тестирования программа успешно обработала все граничные условия.

### 3.2.3 Причинно-следственная диаграмма

Для построения причинно-следственной диаграммы введём следующие обозначения.

#### Причины:

1. Ввод является целым числом.
2. Число  $n \leq 1$ .
3. Число  $n = 2$ .
4. Число  $n$  чётное и  $n \neq 2$ .
5. Среди нечётных чисел от 3 до  $\lfloor \sqrt{n} \rfloor$  найден делитель  $n$ .

#### Следствия:

21. Программа выводит сообщение об ошибке ввода и запрашивает повторный ввод.
22. Программа выводит результат: **не простое** (по условию  $n \leq 1$ ).
23. Программа выводит результат: **простое** (граничный случай  $n = 2$ ).
24. Программа выводит результат: **не простое** (чётное число  $n \neq 2$ ).

25. Программа выводит результат: **не простое** (найден делитель при переборе).

26. Программа выводит результат: **простое** (делитель не найден).

### Промежуточные узлы:

11. Причина 1 истинна И причина 2 ложна (корректный ввод,  $n > 1$ ).

12. Узел 11 истинен И причина 3 ложна И причина 4 ложна (число нечётное  $> 2$ , выполняется перебор).

Таблица 4: Таблица решений (метод чёрного ящика)

|                                                      | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|------------------------------------------------------|---|---|---|---|---|---|---|
| Причина 1 (целое число)                              | 0 | 1 | 1 | 1 | 1 | 1 | 1 |
| Причина 2 ( $n \leq 1$ )                             | – | 1 | 0 | 0 | 0 | 0 | 0 |
| Причина 3 ( $n = 2$ )                                | – | – | – | 1 | 0 | 0 | 0 |
| Причина 4 ( $n$ чётное, $\neq 2$ )                   | – | – | – | – | 1 | 0 | 0 |
| Причина 5 (делитель найден)                          | – | – | – | – | – | 1 | 0 |
| Узел 11 ( $C_1 \wedge \neg C_2$ )                    | 0 | 0 | 1 | 1 | 1 | 1 | 1 |
| Узел 12 ( $U_{11} \wedge \neg C_3 \wedge \neg C_4$ ) | 0 | 0 | 0 | 0 | 0 | 1 | 1 |
| Следствие 21 (ошибка ввода)                          | 1 | 0 | 0 | 0 | 0 | 0 | 0 |
| Следствие 22 ( <b>не простое</b> , $n \leq 1$ )      | 0 | 1 | 0 | 0 | 0 | 0 | 0 |
| Следствие 23 ( <b>простое</b> , $n = 2$ )            | 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| Следствие 24 ( <b>не простое</b> , чётное)           | 0 | 0 | 0 | 0 | 1 | 0 | 0 |
| Следствие 25 ( <b>не простое</b> , делитель найден)  | 0 | 0 | 0 | 0 | 0 | 1 | 0 |
| Следствие 26 ( <b>простое</b> , делитель не найден)  | 0 | 0 | 0 | 0 | 0 | 0 | 1 |

Таблица 5: Тесты на основе таблицы решений

| N | Комбинация                  | n   | Ожидаемый рез.                  | Фактич. рез.                    | Результат |
|---|-----------------------------|-----|---------------------------------|---------------------------------|-----------|
| 1 | $C_1 = 0$                   | abc | Следствие 21: ошибка ввода      | Следствие 21: ошибка ввода      | Пройден   |
| 2 | $C_1 = 1, C_2 = 1$          | -3  | Следствие 22: <b>не простое</b> | Следствие 22: <b>не простое</b> | Пройден   |
| 3 | $C_1 = 1, C_2 = 0, C_3 = 1$ | 2   | Следствие 23: <b>простое</b>    | Следствие 23: <b>простое</b>    | Пройден   |
| 4 | $C_1 = 1, C_2 = 0, C_4 = 1$ | 8   | Следствие 24: <b>не простое</b> | Следствие 24: <b>не простое</b> | Пройден   |
| 5 | $C_1 = 1, C_5 = 1$          | 15  | Следствие 25: <b>не простое</b> | Следствие 25: <b>не простое</b> | Пройден   |
| 6 | $C_1 = 1, C_5 = 0$          | 17  | Следствие 26: <b>простое</b>    | Следствие 26: <b>простое</b>    | Пройден   |

### 3.2.4 Результаты тестирования программы №1 методом чёрного ящика

В результате тестирования методом чёрного ящика было составлено 26 тестов (12 классов эквивалентности + 8 граничных значений + 6 тестов по таблице решений). Все тесты были успешно пройдены, дефектов обнаружено не было.

## 3.3 Тестирование методом «белого» ящика

Алгоритм составления тестов методом «белого» ящика предполагает обход всех возможных путей в теле программы и проверку выполнения каждого оператора не менее одного раза.

В программе можно выделить следующие ключевые условия:

1. Условие C1: ввод является корректным целым числом.  
Ветвь F (False): вывод ошибки ввода, повторный запрос.  
Ветвь T (True): продолжение выполнения.
2. Условие C2:  $n \leq 1$ .  
Ветвь T (True): вывод «не простое», завершение.  
Ветвь F (False): продолжение.
3. Условие C3:  $n = 2$ .  
Ветвь T (True): вывод «простое», завершение.  
Ветвь F (False): продолжение.
4. Условие C4:  $n$  чётное и  $n \neq 2$ .  
Ветвь T (True): вывод «не простое», завершение.  
Ветвь F (False): переход к перебору делителей.
5. Условие C5: в теле цикла  $i \leq \lfloor \sqrt{n} \rfloor$  (условие продолжения цикла).  
Ветвь T (True): продолжение итераций.  
Ветвь F (False): выход из цикла.
6. Условие C6:  $n \bmod i = 0$  (найден делитель).  
Ветвь T (True): вывод «не простое», завершение.  
Ветвь F (False): переход к следующей итерации.

### Возможные пути выполнения:

1. Некорректный ввод: C1(F)  $\rightarrow$  повторный ввод.

2.  $n \leq 1$ :  $C1(T) \rightarrow C2(T) \rightarrow$  «не простое».
3.  $n = 2$ :  $C1(T) \rightarrow C2(F) \rightarrow C3(T) \rightarrow$  «простое».
4. Чётное  $n > 2$ :  $C1(T) \rightarrow C2(F) \rightarrow C3(F) \rightarrow C4(T) \rightarrow$  «не простое».
5. Нечётное составное:  $C1(T) \rightarrow \dots \rightarrow C4(F) \rightarrow$  цикл  $\rightarrow C5(T) \rightarrow C6(T) \rightarrow$  «не простое».
6. Нечётное простое:  $C1(T) \rightarrow \dots \rightarrow C4(F) \rightarrow$  цикл  $\rightarrow C5(T) \rightarrow C6(F)$  повторяется  $\rightarrow C5(F) \rightarrow$  «простое».

### 3.3.1 Покрытие операторов

Для выполнения каждого оператора хотя бы один раз необходим тест, проходящий по основному пути (нечётное простое число), при котором все операторы цикла выполняются.

Таблица 6: Покрытие операторов

| N | n  | Ожидаемый результат | Фактический результат | Статус  |
|---|----|---------------------|-----------------------|---------|
| 1 | 17 | простое             | простое               | Пройден |

Тест №1 обеспечивает выполнение всех операторов программы: ввод, проверки C2–C4, цикл с проверкой C5 и C6, вывод результата.

### 3.3.2 Покрытие решений

Необходимо, чтобы каждое из условий C1–C6 приняло оба значения (True и False).

Таблица 7: Покрытие решений

| N | n   | Ожид. рез.   | Факт. рез.   | C1 | C2 | C3 | C4 | C5  | C6 | Статус  |
|---|-----|--------------|--------------|----|----|----|----|-----|----|---------|
| 1 | 17  | простое      | простое      | T  | F  | F  | F  | T/F | F  | Пройден |
| 2 | abc | ошибка ввода | ошибка ввода | F  | –  | –  | –  | –   | –  | Пройден |
| 3 | 0   | не простое   | не простое   | T  | T  | –  | –  | –   | –  | Пройден |
| 4 | 2   | простое      | простое      | T  | F  | T  | –  | –   | –  | Пройден |
| 5 | 8   | не простое   | не простое   | T  | F  | F  | T  | –   | –  | Пройден |
| 6 | 15  | не простое   | не простое   | T  | F  | F  | F  | T   | T  | Пройден |

Тесты №1 и №6 покрывают оба значения C5 и C6. Все условия протестированы на T и F.

### 3.3.3 Покрытие условий

Каждое элементарное условие должно принять значения True и False хотя бы по одному разу. Условия C1–C6 являются элементарными. Набор тестов из предыдущего раздела полностью удовлетворяет данному критерию.

Таблица 8: Покрывание условий

| N | n   | Ожид. рез.   | Факт. рез.   | C1 | C2 | Статус  |
|---|-----|--------------|--------------|----|----|---------|
| 1 | 17  | простое      | простое      | T  | F  | Пройден |
| 2 | abc | ошибка ввода | ошибка ввода | F  | –  | Пройден |
| 3 | 0   | не простое   | не простое   | T  | T  | Пройден |

Все элементарные условия C1 и C2 проверены на оба логических значения. Условия C3–C6 аналогично покрыты тестами из таблицы покрытия решений.

### 3.3.4 Покрывание решений и условий

Критерий выполнен набором тестов, составленным для покрытия решений: каждое из условий C1–C6 принимает оба значения, все исходы ветвлений достигнуты хотя бы однократно.

Таблица 9: Покрывание решений и условий

| N | n   | Ожид. рез.   | Факт. рез.   | C1 | C2 | C3 | C4 | C5  | C6 | Статус  |
|---|-----|--------------|--------------|----|----|----|----|-----|----|---------|
| 1 | 17  | простое      | простое      | T  | F  | F  | F  | T/F | F  | Пройден |
| 2 | abc | ошибка ввода | ошибка ввода | F  | –  | –  | –  | –   | –  | Пройден |
| 3 | 0   | не простое   | не простое   | T  | T  | –  | –  | –   | –  | Пройден |
| 4 | 2   | простое      | простое      | T  | F  | T  | –  | –   | –  | Пройден |
| 5 | 8   | не простое   | не простое   | T  | F  | F  | T  | –   | –  | Пройден |
| 6 | 15  | не простое   | не простое   | T  | F  | F  | F  | T   | T  | Пройден |

Все решения и условия покрыты.

### 3.3.5 Комбинаторное покрытие условий

Данный критерий требует проверки всех достижимых комбинаций значений элементарных условий. Поскольку условия проверяются последовательно (при истинности предыдущего условие-прерыватель не вычисляет следующее), число достижимых комбинаций ограничено структурой программы.

1. C1=F: программа немедленно запрашивает повторный ввод; C2–C6 недостижимы.
2. C1=T, C2=T: программа выводит «не простое»; C3–C6 недостижимы.
3. C1=T, C2=F, C3=T: программа выводит «простое»; C4–C6 недостижимы.
4. C1=T, C2=F, C3=F, C4=T: программа выводит «не простое»; C5, C6 недостижимы.

5.  $C1=T, C2=F, C3=F, C4=F, C5=T, C6=T$ : делитель найден  $\rightarrow$  «не простое».
6.  $C1=T, C2=F, C3=F, C4=F, C5=T, C6=F$ : делитель не найден на данной итерации  $\rightarrow$  следующая итерация.
7.  $C1=T, C2=F, C3=F, C4=F, C5=F$ : цикл завершён без нахождения делителя  $\rightarrow$  «простое».

Таблица 10: Комбинаторное покрытие условий

| N | n   | Ожид. рез.   | Факт. рез.   | C1 | C2 | C3 | C4 | C5  | C6 | Статус  |
|---|-----|--------------|--------------|----|----|----|----|-----|----|---------|
| 1 | abc | ошибка ввода | ошибка ввода | F  | –  | –  | –  | –   | –  | Пройден |
| 2 | 0   | не простое   | не простое   | T  | T  | –  | –  | –   | –  | Пройден |
| 3 | 2   | простое      | простое      | T  | F  | T  | –  | –   | –  | Пройден |
| 4 | 8   | не простое   | не простое   | T  | F  | F  | T  | –   | –  | Пройден |
| 5 | 15  | не простое   | не простое   | T  | F  | F  | F  | T   | T  | Пройден |
| 6 | 17  | простое      | простое      | T  | F  | F  | F  | T/F | F  | Пройден |

Все достижимые комбинации условий покрыты. Тест №6 охватывает одновременно комбинации (5), (6) и (7), поскольку при  $n = 17$  цикл выполняет несколько итераций с  $C6=F$ , а затем  $C5$  принимает значение F.

### 3.3.6 Результаты тестирования программы №1 методом белого ящика

В ходе тестирования методом белого ящика были разработаны тесты для пяти критериев покрытия кода.

Для покрытия операторов достаточно одного теста ( $n = 17$ ): он проходит по пути  $C1(T) \rightarrow C2(F) \rightarrow C3(F) \rightarrow C4(F) \rightarrow \text{цикл} \rightarrow \text{вывод «простое»}$  и охватывает все операторы программы.

Для покрытия решений потребовалось шесть тестов, которые в совокупности обеспечивают обе ветви каждого из условий  $C1$ – $C6$ . Аналогичный набор тестов удовлетворяет и критерию покрытия условий, поскольку каждое из условий  $C1$ – $C6$  является элементарным.

Покрытие решений и условий достигается тем же набором из шести тестов.

Для комбинаторного покрытия выявлено семь достижимых комбинаций (все остальные комбинации недостижимы из-за последовательного характера ветвлений). Шесть тестов покрывают все семь достижимых комбинаций. Все тесты пройдены успешно.



## 4 Тестирование программы №2

### 4.1 Описание программы

**Автор:** Попов Илья.

**Название:** алгоритм подсчета вхождений подстроки в массиве строк.

**Дано:**

1. `arr` — массив строк.
2. `substr` — подстрока для поиска в массиве.

**Требуется:** найти количество вхождений `count` подстроки в массиве строк.

**Ограничения:**

1. Длина  $n$  подстроки `substr`:  $n \in [0; 1000]$
2. Длина  $n$  строки `str` в массиве `arr`:  $n \in [1; 1000]$
3. Размер массива  $n \in [1; 100]$
4. `count` — целое неотрицательное число.

**Спецификация:**

| Входные данные                                                            | Выходные данные                           | Реакция программы                                                                |
|---------------------------------------------------------------------------|-------------------------------------------|----------------------------------------------------------------------------------|
| <code>arr = {"hello "world "hello"}</code><br><code>substr = "he"</code>  | <code>count = 2</code>                    | Подстрока встречается дважды. Возвращается значение. Программа завершает работу. |
| <code>arr = {"hello "world "hello"}</code><br><code>substr =</code>       | Error: "Подстрока не может быть пустой"   | Программа генерирует исключение. Завершение работы программы.                    |
| <code>arr = {"hello "world "hello"}</code><br><code>substr = "eee"</code> | <code>count = 0</code>                    | Подстрока не встречается. Возвращается значение. Программа завершает работу.     |
| <code>arr = {}</code><br><code>substr = "he"</code>                       | Error: "Массив пустой"                    | Программа генерирует исключение. Завершение работы программы.                    |
| длина <code>arr</code> > 100<br><code>substr = "e"</code>                 | Error: "Массив слишком большой"           | Программа генерирует исключение. Завершение работы программы.                    |
| длина элемента в <code>arr</code> > 1000<br><code>substr = "e"</code>     | Error: "Слишком длинная строка в массиве" | Программа генерирует исключение. Завершение работы программы.                    |

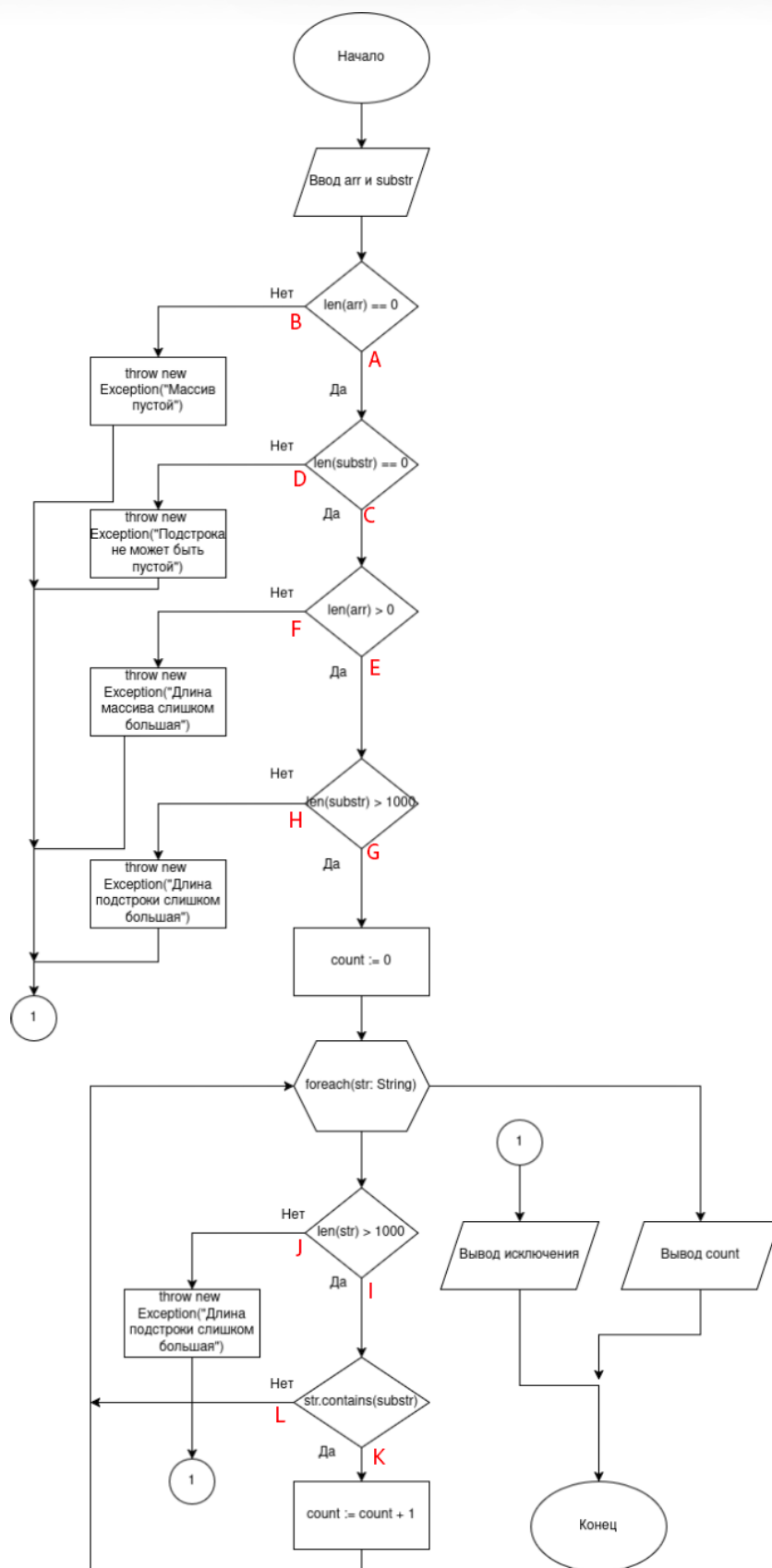


Рис. 2: Блок-схема программы №2

## 4.2 Тестирование методом «чёрного» ящика

### 4.2.1 Разбиение на классы эквивалентности

Исходя из ограничений программы составлено разбиение на классы эквивалентности по двум входным параметрам: `arr` и `substr`.

Таблица 11: Классы эквивалентности для программы №2

| Входное условие               | Допустимые классы                                           | Недопустимые классы                           |
|-------------------------------|-------------------------------------------------------------|-----------------------------------------------|
| Массив <code>arr</code>       | Массив из 1 элемента (1)                                    | Пустой массив (7)                             |
|                               | Массив из 2–99 элементов (2)                                | Массив из более чем 100 элементов (8)         |
|                               | Массив ровно из 100 элементов (3)                           | Массив, содержащий строку длиной $> 1000$ (9) |
| Подстрока <code>substr</code> | Непустая подстрока, встречающаяся в <code>arr</code> (4)    | Пустая строка (10)                            |
|                               | Непустая подстрока, не встречающаяся в <code>arr</code> (5) | Подстрока длиной $> 1000$ (11)                |
|                               | Подстрока длиной ровно 1000 (6)                             |                                               |

Таблица 12: Тесты по классам эквивалентности

| N  | <code>arr</code>                         | <code>substr</code>          | Ожидаемый результат               | Фактический результат             | Результат |
|----|------------------------------------------|------------------------------|-----------------------------------|-----------------------------------|-----------|
| 1  | <code>{"hello"}</code>                   | <code>"he"</code>            | <code>count = 1</code>            | <code>count = 1</code>            | Пройден   |
| 2  | <code>{"hello "world "hi"}</code>        | <code>"h "</code>            | <code>count = 2</code>            | <code>count = 2</code>            | Пройден   |
| 3  | Массив из 100 элементов <code>"a"</code> | <code>"a"</code>             | <code>count = 100</code>          | <code>count = 100</code>          | Пройден   |
| 4  | <code>{"hello "world"}</code>            | <code>"world"</code>         | <code>count = 1</code>            | <code>count = 1</code>            | Пройден   |
| 5  | <code>{"hello "world"}</code>            | <code>"xyz"</code>           | <code>count = 0</code>            | <code>count = 0</code>            | Пройден   |
| 6  | <code>{"hello"}</code>                   | <code>substr</code> дл. 1000 | <code>count = 0</code>            | <code>count = 0</code>            | Пройден   |
| 7  | <code>{}</code>                          | <code>"he"</code>            | Ошибка: массив пустой             | Ошибка: массив пустой             | Пройден   |
| 8  | Массив из 101 эл.                        | <code>"a"</code>             | Ошибка: массив слишком большой    | Ошибка: массив слишком большой    | Пройден   |
| 9  | <code>{"hello str дл. 1001"}</code>      | <code>"he"</code>            | Ошибка: строка слишком длинная    | Ошибка: строка слишком длинная    | Пройден   |
| 10 | <code>{"hello"}</code>                   |                              | Ошибка: пустая подстрока          | Ошибка: пустая подстрока          | Пройден   |
| 11 | <code>{"hello"}</code>                   | <code>substr</code> дл. 1001 | Ошибка: подстрока слишком длинная | Ошибка: подстрока слишком длинная | Пройден   |

Все 11 тестов прошли успешно.

## 4.2.2 Анализ граничных условий

Исходя из ограничений программы выделяются следующие граничные значения:

1. Размер массива: 0 (пустой), 1 (минимальный допустимый), 100 (максимальный допустимый), 101 (первый недопустимый).
2. Длина строки в массиве: 1 (минимальная), 1000 (максимальная), 1001 (первая недопустимая).
3. Длина подстроки: 0 (пустая — недопустима), 1 (минимальная допустимая), 1000 (максимальная), 1001 (первая недопустимая).
4. Количество совпадений: 0 (нет вхождений), 1, все элементы.

Таблица 13: Проверка граничных условий программы №2

| N  | arr                    | substr          | Ожидаемый результат               | Фактический результат             | Результат |
|----|------------------------|-----------------|-----------------------------------|-----------------------------------|-----------|
| 1  | {}                     | "a"             | Ошибка: массив пустой             | Ошибка: массив пустой             | Пройден   |
| 2  | {"a"}                  | "a"             | count = 1                         | count = 1                         | Пройден   |
| 3  | Массив из 100 эл. "a"  | "a"             | count = 100                       | count = 100                       | Пройден   |
| 4  | Массив из 101 эл.      | "a"             | Ошибка: массив слишком большой    | Ошибка: массив слишком большой    | Пройден   |
| 5  | {"a"} (дл. строки = 1) | "a"             | count = 1                         | count = 1                         | Пройден   |
| 6  | {str дл. 1000}         | "a"             | count = 1000 или 0                | соответствует                     | Пройден   |
| 7  | {str дл. 1001}         | "a"             | Ошибка: строка слишком длинная    | Ошибка: строка слишком длинная    | Пройден   |
| 8  | {"hello"}              |                 | Ошибка: пустая подстрока          | Ошибка: пустая подстрока          | Пройден   |
| 9  | {"hello"}              | "h"(дл. 1)      | count = 1                         | count = 1                         | Пройден   |
| 10 | {"hello"}              | substr дл. 1000 | count = 0                         | count = 0                         | Пройден   |
| 11 | {"hello"}              | substr дл. 1001 | Ошибка: подстрока слишком длинная | Ошибка: подстрока слишком длинная | Пройден   |
| 12 | {"a "b "c"}            | "x"             | count = 0                         | count = 0                         | Пройден   |
| 13 | {"a "a "a"}            | "a"             | count = 3 (все)                   | count = 3                         | Пройден   |

Все 13 тестов прошли успешно.

### 4.2.3 Причинно-следственная диаграмма

Для построения причинно-следственной диаграммы введём следующие обозначения.

#### Причины:

1. Массив `arr` пустой ( $\text{len}(\text{arr}) = 0$ ).
2. Подстрока `substr` пустая ( $\text{len}(\text{substr}) = 0$ ).
3. Массив слишком большой ( $\text{len}(\text{arr}) > 100$ ).
4. Подстрока слишком длинная ( $\text{len}(\text{substr}) > 1000$ ).
5. Строка в массиве слишком длинная ( $\exists \text{str} \in \text{arr} : \text{len}(\text{str}) > 1000$ ).
6. Подстрока встречается хотя бы в одном элементе массива ( $\text{count} > 0$ ).

#### Следствия:

21. Ошибка: «Массив пустой».
22. Ошибка: «Подстрока не может быть пустой».
23. Ошибка: «Длина массива слишком большая».
24. Ошибка: «Длина подстроки слишком большая».
25. Ошибка: «Длина строки слишком большая».
26. Вывод  $\text{count} > 0$  (есть совпадения).
27. Вывод  $\text{count} = 0$  (нет совпадений).

#### Ограничения:

- $E(1, 3)$ : причины 1 и 3 взаимоисключающие (пустой массив не может одновременно быть слишком большим).
- При  $C1=T$  программа завершается до проверки  $C2-C6$ .

Таблица 14: Таблица решений (причинно-следственная диаграмма)

|                                      | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|--------------------------------------|---|---|---|---|---|---|---|
| Причина 1 (arr пустой)               | 1 | 0 | 0 | 0 | 0 | 0 | 0 |
| Причина 2 (substr пустая)            | – | 1 | 0 | 0 | 0 | 0 | 0 |
| Причина 3 (arr > 100)                | – | – | 1 | 0 | 0 | 0 | 0 |
| Причина 4 (substr > 1000)            | – | – | – | 1 | 0 | 0 | 0 |
| Причина 5 (строка > 1000)            | – | – | – | – | 1 | 0 | 0 |
| Причина 6 (count > 0)                | – | – | – | – | – | 1 | 0 |
| Следствие 21 (ошибка: arr пуст)      | 1 | 0 | 0 | 0 | 0 | 0 | 0 |
| Следствие 22 (ошибка: substr пуста)  | 0 | 1 | 0 | 0 | 0 | 0 | 0 |
| Следствие 23 (ошибка: arr > 100)     | 0 | 0 | 1 | 0 | 0 | 0 | 0 |
| Следствие 24 (ошибка: substr > 1000) | 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| Следствие 25 (ошибка: строка > 1000) | 0 | 0 | 0 | 0 | 1 | 0 | 0 |
| Следствие 26 (count > 0)             | 0 | 0 | 0 | 0 | 0 | 1 | 0 |
| Следствие 27 (count = 0)             | 0 | 0 | 0 | 0 | 0 | 0 | 1 |

Таблица 15: Тесты на основе таблицы решений

| N | arr                    | substr          | Ожидаемый рез.                          | Фактич. рез.                         | Результат |
|---|------------------------|-----------------|-----------------------------------------|--------------------------------------|-----------|
| 1 | {}                     | "he"            | Следствие 21: ошибка «массив пустой»    | Следствие 21: ошибка «массив пустой» | Пройден   |
| 2 | {"hello"}              |                 | Следствие 22: ошибка «пустая подстрока» | Следствие 22: ошибка                 | Пройден   |
| 3 | Массив из 101 эл.      | "a"             | Следствие 23: ошибка «arr > 100»        | Следствие 23: ошибка                 | Пройден   |
| 4 | {"hello"}              | substr дл. 1001 | Следствие 24: ошибка «substr > 1000»    | Следствие 24: ошибка                 | Пройден   |
| 5 | {"hello str" дл. 1001} | "he"            | Следствие 25: ошибка «строка > 1000»    | Следствие 25: ошибка                 | Пройден   |
| 6 | {"hello "world"}       | "hello"         | Следствие 26: count = 1                 | Следствие 26: count = 1              | Пройден   |
| 7 | {"cat "dog"}           | "xyz"           | Следствие 27: count = 0                 | Следствие 27: count = 0              | Пройден   |

Все 7 тестов прошли успешно.

#### 4.2.4 Результаты тестирования программы №2 методом чёрного ящика

В результате тестирования методом чёрного ящика было составлено 31 тест (11 тестов по классам эквивалентности + 13 граничных условий + 7 тестов по таблице решений). Все тесты прошли успешно.

Тестирование по классам эквивалентности охватило все выделенные допустимые и недопустимые классы для входных параметров `arr` и `substr`. Анализ граничных условий выявил корректность обработки крайних значений по всем трём измерениям: размеру массива, длине строки и длине подстроки. Тестирование на основе таблицы решений верифицировало каждое следствие в отдельности.

### 4.3 Тестирование методом белого ящика

Алгоритм составления тестов методом «белого» ящика предполагает обход всех возможных путей в теле программы и проверку выполнения каждого оператора не менее одного раза.

В программе можно выделить следующие ключевые условия и соответствующие им ветви:

1. Условие C1: (`len(arr) == 0`):

Ветвь T (True): Выход из программы с ошибкой «Массив пустой».

Ветвь F (False): Продолжение выполнения.

2. Условие C2: (`len(substr) == 0`):

Ветвь T (True): Выход из программы с ошибкой «Подстрока не может быть пустой».

Ветвь F (False): Продолжение выполнения.

3. Условие C3: (`len(arr) > 100`):

Ветвь T (True): Выход из программы с ошибкой «Длина массива слишком большая».

Ветвь F (False): Продолжение выполнения.

4. Условие C4: (`len(substr) > 1000`):

Ветвь T (True): Выход из программы с ошибкой «Длина подстроки слишком большая».

Ветвь F (False): Продолжение выполнения.

5. Условие C5: (`len(str) > 1000`) (внутри цикла `foreach`):

Ветвь T (True): Выход из программы с ошибкой «Длина строки слишком большая».

Ветвь F (False): Продолжение проверки или выход из цикла.

6. Условие C6: (`str.contains(substr)`):

Ветвь T (True): Увеличение счётчика `count`.

Ветвь F (False): Переход к следующему элементу.

### Возможные пути выполнения:

1. Пустой массив:  $C1(T) \rightarrow \text{Завершение}$ .
2. Пустая подстрока:  $C1(F) \rightarrow C2(T) \rightarrow \text{Завершение}$ .
3. Массив слишком большой:  $C1(F) \rightarrow C2(F) \rightarrow C3(T) \rightarrow \text{Завершение}$ .
4. Подстрока слишком длинная:  $C1(F) \rightarrow C2(F) \rightarrow C3(F) \rightarrow C4(T) \rightarrow \text{Завершение}$ .
5. Строка в массиве слишком длинная:  $C1(F) \rightarrow \dots \rightarrow C4(F) \rightarrow \text{цикл} \rightarrow C5(T) \rightarrow \text{Завершение}$ .
6. Успешный поиск (есть совпадения):  $C1(F) \rightarrow \dots \rightarrow C4(F) \rightarrow \text{цикл} \rightarrow C5(F) \rightarrow C6(T) \rightarrow \dots \rightarrow \text{Вывод count}$ .
7. Поиск без совпадений:  $C1(F) \rightarrow \dots \rightarrow C4(F) \rightarrow \text{цикл} \rightarrow C5(F) \rightarrow C6(F) \rightarrow \dots \rightarrow \text{Вывод count} = 0$ .
8. Смешанный поиск:  $C1(F) \rightarrow \dots \rightarrow C4(F) \rightarrow \text{цикл с многократным } C5(F), C6(T/F) \rightarrow \text{Вывод count}$ .

#### 4.3.1 Покрытие операторов

Для покрытия всех операторов достаточно одного теста, проходящего по пути с успешным поиском.

Таблица 16: Покрытие операторов

| N | arr                     | substr | Ожид. результат | Факт. результат | Статус  |
|---|-------------------------|--------|-----------------|-----------------|---------|
| 1 | {"hello "world "hello"} | "he"   | 2               | 2               | Пройден |

Тест №1 выполнил все операторы программы.

#### 4.3.2 Покрытие решений

Все условия были протестированы на T и F.



Таблица 17: Покрытие решений

| N | arr                    | substr          | Ожид. рез.                        | Факт. рез.                        | C1 | C2 | C3 | C4 | C5 | C6 | Статус  |
|---|------------------------|-----------------|-----------------------------------|-----------------------------------|----|----|----|----|----|----|---------|
| 1 | {"hello "world"}       | "hello"         | 2                                 | 2                                 | F  | F  | F  | F  | F  | T  | Пройден |
| 2 | {"cat "dog"}{"xyz"}    | "xyz"           | 0                                 | 0                                 | F  | F  | F  | F  | F  | F  | Пройден |
| 3 | {}                     | "he"            | Ошибка: массив пустой             | Ошибка: массив пустой             | T  | –  | –  | –  | –  | –  | Пройден |
| 4 | {"hello"}              |                 | Ошибка: пустая подстрока          | Ошибка: пустая подстрока          | F  | T  | –  | –  | –  | –  | Пройден |
| 5 | arr из 101 эл.         | "a"             | Ошибка: массив слишком большой    | Ошибка: массив слишком большой    | F  | F  | T  | –  | –  | –  | Пройден |
| 6 | {"hello"}              | substr дл. 1001 | Ошибка: подстрока слишком длинная | Ошибка: подстрока слишком длинная | F  | F  | F  | T  | –  | –  | Пройден |
| 7 | {"hello str дл. 1001}" | "he"            | Ошибка: строка слишком длинная    | Ошибка: строка слишком длинная    | F  | F  | F  | F  | T  | –  | Пройден |

Таблица 18: Покрытие условий

| N | arr              | substr | Ожид. рез.               | Факт. рез.               | C1 | C2 | Статус  |
|---|------------------|--------|--------------------------|--------------------------|----|----|---------|
| 1 | {"hello "world"} | "he"   | 1                        | 1                        | F  | F  | Пройден |
| 2 | {}               | "he"   | Ошибка: массив пустой    | Ошибка: массив пустой    | T  | –  | Пройден |
| 3 | {"hello"}        |        | Ошибка: пустая подстрока | Ошибка: пустая подстрока | F  | T  | Пройден |

### 4.3.3 Покрытие условий

Все элементарные условия были проверены на T и F.

### 4.3.4 Покрытие решений и условий

Тесты, написанные для покрытия решений, обеспечивают и покрытие решений и условий.

Все решения и условия покрыты.

### 4.3.5 Комбинаторное покрытие условий

Для условий C1 и C2 возможны комбинации (F,F), (T,–), (F,T). Комбинация (T,T) недостижима, так как при C1=T программа завершается до проверки C2.

Таблица 19: Покрывание решений и условий

| N | arr                     | substr          | Ож. рез.                          | Факт. рез.                        | C1 | C2 | C3 | C4 | C5 | C6 | Статус  |
|---|-------------------------|-----------------|-----------------------------------|-----------------------------------|----|----|----|----|----|----|---------|
| 1 | {"hello "world "hello"} | "hello"         | 2                                 | 2                                 | F  | F  | F  | F  | F  | T  | Пройден |
| 2 | {"cat "dog"} "xyz"      | "xyz"           | 0                                 | 0                                 | F  | F  | F  | F  | F  | F  | Пройден |
| 3 | {}                      | "he"            | Ошибка: массив пустой             | Ошибка: массив пустой             | T  | –  | –  | –  | –  | –  | Пройден |
| 4 | {"hello"}               |                 | Ошибка: пустая подстрока          | Ошибка: пустая подстрока          | F  | T  | –  | –  | –  | –  | Пройден |
| 5 | arr из 101 эл.          | "a"             | Ошибка: массив слишком большой    | Ошибка: массив слишком большой    | F  | F  | T  | –  | –  | –  | Пройден |
| 6 | {"hello"}               | substr дл. 1001 | Ошибка: подстрока слишком длинная | Ошибка: подстрока слишком длинная | F  | F  | F  | T  | –  | –  | Пройден |
| 7 | {"hello str дл. 1001}   | "he"            | Ошибка: строка слишком длинная    | Ошибка: строка слишком длинная    | F  | F  | F  | F  | T  | –  | Пройден |

Таблица 20: Комбинаторное покрытие условий

| N | arr                     | substr          | Ожид. рез.                        | Факт. рез.                        | C1 | C2       | Статус  |
|---|-------------------------|-----------------|-----------------------------------|-----------------------------------|----|----------|---------|
| 1 | {"hello "world "hello"} | "hello"         | 2                                 | 2                                 | F  | F        | Пройден |
| 2 | {}                      | "he"            | Ошибка: массив пустой             | Ошибка: массив пустой             | T  | –        | Пройден |
| 3 | {"hello"}               |                 | Ошибка: пустая подстрока          | Ошибка: пустая подстрока          | F  | T        | Пройден |
| 4 | arr из 101 эл.          | "a"             | Ошибка: массив слишком большой    | Ошибка: массив слишком большой    | F  | F (C3=T) | Пройден |
| 5 | {"hello"}               | substr дл. 1001 | Ошибка: подстрока слишком длинная | Ошибка: подстрока слишком длинная | F  | F (C4=T) | Пройден |
| 6 | {"hello str дл. 1001}   | "he"            | Ошибка: строка слишком длинная    | Ошибка: строка слишком длинная    | F  | F (C5=T) | Пройден |
| 7 | {"cat "dog"} "xyz"      | "xyz"           | 0                                 | 0                                 | F  | F (C6=F) | Пройден |

Все достижимые комбинации условий покрыты. Комбинация C1=T и C2=T недостижима.

#### 4.3.6 Результаты тестирования программы №2 методом белого ящика

В ходе тестирования алгоритма подсчёта вхождений подстроки в массиве строк методом белого ящика были разработаны тесты для пяти

критериев покрытия кода. Все тесты пройдены успешно, ошибок в логике программы обнаружено не было. Подробные результаты приведены в таблицах выше.

## 5 Заключение

В ходе выполнения данной лабораторной работы были изучены и применены методы тестирования «белым ящиком» и «чёрным ящиком» для обеих программ.

**Программа №1 (проверка простоты числа).** Тестирование методом «чёрного ящика» включало 26 тестов: 12 по классам эквивалентности, 8 по граничным значениям и 6 по таблице решений. Все тесты пройдены успешно. Тестирование методом «белого ящика» включало 6 тестов, охватывающих пять критериев покрытия (операторов, решений, условий, решений и условий, комбинаторное). Все 7 достижимых комбинаций условий C1–C6 покрыты. Дефектов обнаружено не было.

**Программа №2 (подсчёт вхождений подстроки).** Тестирование методом «чёрного ящика» включало 31 тест: 11 по классам эквивалентности, 13 по граничным значениям и 7 по таблице решений. Все тесты пройдены успешно. Тестирование методом «белого ящика» включало 7 тестов, охватывающих пять критериев покрытия кода. Комбинация C1=T, C2=T признана недостижимой вследствие раннего завершения программы при C1=T. Все тесты пройдены успешно.

По итогам выполненной работы можно сделать вывод, что оба метода тестирования дополняют друг друга и в совокупности обеспечивают высокое качество проверки программного обеспечения. Метод «белого ящика» позволяет убедиться в корректности внутренней логики и полноте охвата всех ветвей кода. Метод «чёрного ящика» проверяет соответствие поведения программы требованиям спецификации без учёта внутренней реализации. Совместное применение обоих методов снижает вероятность пропуска дефектов и обеспечивает комплексную верификацию программного продукта.

# Список литературы

- [1] Майерс Г. «Искусство тестирования программ», СПб, Диалектика, 2012г.